

COUNCILia

Persistencia de Chats

Análisis de arquitectura para almacenar conversaciones.

- Contexto del problema
- Idea 1: Chat completo en Supabase
- Idea 2: Solo metadata y resumen
- Idea 3: Híbrido localStorage + sync
- Comparativa y costos
- Recomendación para el MVP

Tabla de contenidos

Contexto del problema

Idea 1: Chat completo en Supabase (texto + contexto)

Idea 2: Solo metadata y resumen de contexto

Idea 3: Híbrido — localStorage + sincronización lazy

Comparativa rápida

Recomendación para el MVP

Contexto del problema

Actualmente los chats de COUNCILia se almacenan en **localStorage** del navegador. Esto funciona como prototipo, pero tiene limitaciones importantes:

- Si el usuario cambia de dispositivo o borra su caché, pierde todo.
- No hay forma de que los agentes "recuerden" contexto entre sesiones.
- No se puede analizar el uso ni mejorar los prompts con datos reales.

El objetivo es evaluar cómo persistir las conversaciones de forma profesional, escalable y viable para un MVP, usando **Supabase** (PostgreSQL + Auth + Storage).

Idea 1: Chat completo en Supabase (texto + contexto)

Qué guardamos

- **Tabla `chat_sessions`**: id, user_id, title, created_at, updated_at, survey_snapshot (JSON con las respuestas de la encuesta para reconstruir contexto).
- **Tabla `chat_turns`**: id, session_id, turn_number, user_message, agent_responses (JSON con las 3 respuestas), replica (JSON), created_at.
- **Tabla `user_contexts`** (opcional): user_id, extracted_facts (JSON array con datos que los agentes han detectado, p.ej. "ingreso: 10,000/mes").

Cómo lo hacemos

1. Supabase Auth maneja login (email, Google, Apple).
2. Al finalizar cada turno (después de la réplica), un **POST** `/api/chats/save-turn` guarda el turno en `chat_turns`.
3. Antes de enviar el siguiente mensaje al LLM, se consultan los últimos N turnos de la sesión para inyectarlos como contexto en el system prompt.
4. Opcionalmente, un post-proceso extrae "hechos" del texto del usuario (ingreso, situación familiar, etc.) y los guarda en `user_contexts` para reutilizar entre sesiones.

Cuánto cuesta (Supabase)

Concepto	Free Tier	Pro (\$25/mes)
Base de datos	500 MB	8 GB
Auth	50,000 MAU	100,000 MAU
API requests	Ilimitadas	Ilimitadas
Realtime	200 conexiones	500 conexiones

Estimación por usuario: un turno completo (mensaje + 3 respuestas + réplica) pesa ~3-5 KB de texto. Con 20 turnos por sesión y 5 sesiones por usuario: ~500 KB/usuario.

Con 100 usuarios: ~50 MB total. **Cabe holgadamente en el free tier** (500 MB).

Con 1,000 usuarios: ~500 MB. Llegaríamos al límite del free tier; se requeriría el plan Pro (\$25/mes).

Con 10,000 usuarios: ~5 GB. Plan Pro cubre hasta 8 GB.

Ventajas

- Persistencia real: el usuario cambia de dispositivo y encuentra todo.
- Los agentes pueden tener memoria de contexto (inyectando turnos previos o hechos extraídos).
- Se puede analizar el uso, detectar patrones, mejorar prompts.
- Escalable: PostgreSQL aguanta millones de registros sin problema.
- Supabase Auth integrado facilita login social.

Desventajas

- Requiere implementar API routes para CRUD de chats.
- La inyección de contexto al LLM incrementa tokens enviados (costo de Gemini).
- Más complejidad de infraestructura vs. localStorage.
- Los datos del usuario son sensibles; requiere cifrado y política de privacidad robusta.

Idea 2: Solo metadata y resumen de contexto

Qué guardamos

- **Tabla `chat_sessions`:** id, user_id, title, summary (resumen de 2-3 oraciones generado por el LLM al final de cada ciclo), key_facts (JSON con datos clave extraídos), created_at, updated_at.

- **No guardamos** los turnos completos — el texto de las conversaciones vive solo en localStorage como cache temporal.

Cómo lo hacemos

1. Al finalizar un ciclo completo (encuesta → agentes → réplica → acción del usuario), pedimos al LLM un resumen breve de la conversación y los datos clave mencionados.
2. Ese resumen se sube a Supabase como `summary` y `key_facts`.
3. Cuando el usuario abre una nueva sesión, se inyecta el resumen acumulado como contexto: "En sesiones anteriores, el usuario mencionó que gana \$10,000/mes y está preocupado por su relación de pareja."
4. El texto completo de la conversación actual sigue viviendo en localStorage.

Cuánto cuesta

Por usuario: ~1-2 KB por sesión (solo resumen + hechos). Con 5 sesiones: ~10 KB/usuario.

Con 100 usuarios: ~1 MB. **Prácticamente nada.**

Con 1,000 usuarios: ~10 MB.

Con 10,000 usuarios: ~100 MB.

Todo cabe en el free tier de Supabase sin problema.

Costo adicional: ~100-200 tokens extra por ciclo para generar el resumen con el LLM. Insignificante en Gemini Flash.

Ventajas

- Extremadamente ligero en almacenamiento.
- Los agentes obtienen contexto entre sesiones sin inyectar conversaciones enormes.
- Bajo costo de tokens LLM (solo resumen, no historial completo).
- Privacy-friendly: no guardamos texto crudo de conversaciones en el servidor.
- Rápido de implementar: una tabla, un prompt de resumen.

Desventajas

- El usuario **no puede** ver sus conversaciones pasadas completas desde otro dispositivo (solo desde el mismo navegador con localStorage).
- La calidad del contexto depende del resumen del LLM (puede perder matices).
- No se pueden reprocesar conversaciones antiguas para analytics.
- Si el usuario borra localStorage, pierde el historial visible (aunque el resumen persiste).

Idea 3: Híbrido — localStorage + sincronización lazy

Qué guardamos

- **localStorage**: conversación completa en tiempo real (como ahora).
- **Supabase**: backup de las conversaciones cuando el usuario está en wifi/idle, más un resumen de contexto.
- **Tabla chat_sessions**: id, user_id, title, summary, key_facts, full_turns (JSON comprimido, nullable), synced_at.

Cómo lo hacemos

1. La experiencia inmediata sigue siendo localStorage (0 latencia, funciona offline).
2. Después de cada ciclo, en background (con `requestIdleCallback` o un debounce de 5s), sincronizamos el turno a Supabase.
3. Si Supabase no está disponible, se marca como pendiente y se sincroniza en el siguiente ciclo.
4. Al abrir la app en un nuevo dispositivo, se descargan las sesiones de Supabase y se llenan en localStorage.
5. El resumen + key_facts se generan igual que en la Idea 2 para contexto inter-sesión.

Cuánto cuesta

Mismo costo que Idea 1 en almacenamiento (~50 MB para 100 usuarios), pero con mejor UX porque la escritura es async.

Ventajas

- Mejor UX: respuesta instantánea, sin esperar a la red.
- Funciona offline.
- Persistencia completa: conversaciones disponibles en cualquier dispositivo.
- Resumen de contexto para memoria inter-sesión.
- Degradación graceful: si Supabase falla, la app sigue funcionando.

Desventajas

- Mayor complejidad: lógica de sincronización, resolución de conflictos, estado de sync.
- Más código y edge cases que manejar.
- Potenciales inconsistencias si el usuario usa dos dispositivos simultáneamente.
- Requiere más tiempo de desarrollo.

Comparativa rápida

Criterio	Idea 1 (Completo)	Idea 2 (Resumen)	Idea 3 (Híbrido)
Almacenamiento (100 usuarios)	~50 MB	~1 MB	~50 MB
Costo mensual (100 usuarios)	\$0 (free tier)	\$0 (free tier)	\$0 (free tier)
Costo mensual (10k usuarios)	\$25 (Pro)	\$0 (free tier)	\$25 (Pro)
Memoria de contexto	Excelente	Buena	Excelente
Multi-dispositivo	Sí	Parcial	Sí
Complejidad de desarrollo	Media	Baja	Alta
Tiempo estimado	2-3 días	4-6 horas	4-5 días
Funciona offline	No	Parcial	Sí
Analytics posibles	Total	Limitado	Total

Recomendación para el MVP

Fase 1 (MVP inmediato): Idea 2 — Resumen + key_facts

Es la opción más rápida y con mejor relación costo/beneficio:

- Implementación en pocas horas.
- Los agentes ya podrían "recordar" datos clave entre sesiones.
- Costo de infraestructura prácticamente nulo.
- No requiere manejar datos sensibles (texto completo) en el servidor.

Fase 2 (post-lanzamiento): Migrar a Idea 1

Una vez validado el producto con usuarios reales:

- Agregar persistencia completa de turnos.

- Habilitar historial multi-dispositivo.
- Extraer analytics de las conversaciones para mejorar prompts.

Fase 3 (escala): Idea 3

Si se necesita soporte offline o la app evoluciona a móvil:

- Implementar sincronización lazy.
- Agregar resolución de conflictos.

Estructura de tablas recomendada (Supabase)

```
-- Fase 1
CREATE TABLE chat_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  title TEXT NOT NULL DEFAULT 'Nueva sesión',
  summary TEXT,
  key_facts JSONB DEFAULT '[]'::jsonb,
  survey_snapshot JSONB,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Fase 2 (agregar después)
CREATE TABLE chat_turns (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID REFERENCES chat_sessions(id) ON DELETE CASCADE,
  turn_number INT NOT NULL,
  user_message TEXT NOT NULL,
  agent_responses JSONB NOT NULL,
  replica JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS para que cada usuario solo vea sus datos
ALTER TABLE chat_sessions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users see own sessions"
  ON chat_sessions FOR ALL
  USING (auth.uid() = user_id);

ALTER TABLE chat_turns ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users see own turns"
  ON chat_turns FOR ALL
  USING (
    session_id IN (
      SELECT id FROM chat_sessions WHERE user_id = auth.uid()
    )
  );
```


Esta estructura es limpia, escalable, y se puede implementar incrementalmente sin romper nada existente.